SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	P. Lohith Krishna	**Reg. No.:**	192325122
Date:	21/08/2026	**Duration:**	60 minutes

Code of Conduct

I _____P. Lohith Krishna_(192325122)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____P. Lohith Krishna_____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, design a CI/CD (Continuous Integration and Continuous Deployment) pipeline for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. Analyze the project requirements and identify the CI/CD needs of the assigned problem statement.
2. Design the CI/CD pipeline workflow showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable tools and technologies for each stage of the pipeline and justify their selection.
4. Define appropriate automated testing strategies to be integrated into the pipeline.
5. Design the deployment strategy, including development, testing/staging, and production environments.
6. Incorporate appropriate security, quality checks, notifications, and rollback mechanisms in the pipeline.
7. Prepare a CI/CD pipeline diagram and explain the workflow from code commit to deployment.

Deliverable: Submit a CI/CD Pipeline Design document containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Online Food Delivery Management System – CI/CD Pipeline Design Document

1. Project Overview

Project Name: Online Food Delivery Management System

The Online Food Delivery Management System is a web and mobile platform that enables customer registration, restaurant discovery, menu browsing, cart management, online ordering, secure payment, order tracking, delivery assignment, notifications, ratings, and administrative management of restaurants, menus, orders, users, and delivery operations. This CI/CD design ensures frequent and reliable delivery of software updates while maintaining quality, security, performance, and service availability.

2. Analyze Project Requirements and CI/CD Needs

CI/CD is essential for the Online Food Delivery Management System because the platform contains interconnected modules such as ordering, payment, restaurant availability, delivery assignment, and real-time order tracking. Manual builds, testing, and deployment can introduce errors, delay releases, and make recovery from production failures difficult. An automated pipeline reduces these risks by enforcing repeatable builds, automated testing, quality gates, security checks, and controlled deployment.

Core CI/CD Requirements:

- Source code management
- Automated builds
- Automated unit testing
- Integration/API testing
- UI/end-to-end testing
- Code quality analysis
- Security scanning
- Artifact/package creation
- Deployment automation
- Environment management
- Monitoring and notifications
- Rollback capability

3. CI/CD Pipeline Architecture

The automated pipeline transitions application changes through sequential validation and deployment stages:

```
Developer → GitHub Repository → CI → Build
                                  ↓
Unit Tests → Integration/API Tests → Code Quality → Security Scan
                                  ↓
Package/Artifact → Development Deployment → Staging Deployment → Staging Tests
```

```
                                    ↓
Approval → Production Deployment → Monitoring → Rollback if Required
```

Every feature passes source control, build, automated testing, code quality, security, and packaging checks before reaching Development and Staging. Production deployment occurs only after successful staging validation and approval, followed by active monitoring and rollback capability.

4. Tools and Technologies

The following practical technology stack is recommended to implement the CI/CD pipeline efficiently.

Pipeline Stage	Recommended Tool	Justification
Source Code Management	Git + GitHub	Industry-standard version control and collaboration.
CI/CD Automation	GitHub Actions	Native GitHub integration and configuration-as-code.
Backend Build	Maven	Reliable dependency management and Java build automation.
Backend	Java + Spring Boot	Robust framework for scalable service development.
Frontend	React + Bootstrap	Responsive and component-based web interface.
Unit Testing	JUnit	Standard unit testing framework for Java.
API Testing	Postman / Newman	Effective REST API validation in CI pipelines.
Integration Testing	Spring Boot Test	Integrated testing support for Spring applications.
UI Testing	Selenium	Automates browser-based customer journeys.
Code Quality	SonarQube	Detects bugs, code smells, vulnerabilities, and coverage issues.
Dependency/Security Scan	OWASP Dependency-Check	Identifies known dependency vulnerabilities.
Containerization	Docker	Provides consistent environments across stages.
Container Registry	GitHub Container Registry	Secure integration with GitHub Actions.
Deployment	Docker / Docker Compose	Predictable and lightweight deployment.
Monitoring	Prometheus + Grafana	Real-time metrics and visualization.
Notifications	GitHub Actions / Email	Immediate alerts for pipeline events.

5. Pipeline Stages

Stage 1 – Source Code Management

Developers create feature branches, commit changes to Git, and open Pull Requests. Branch protection requires review and successful status checks before merging to the main or develop branch.

Stage 2 – Build

The pipeline checks out the source, configures the required JDK, installs dependencies, and runs the Maven build. Compilation or dependency failures immediately stop the pipeline.

Stage 3 – Automated Testing

Unit, integration, API, UI/end-to-end, and regression tests validate critical workflows such as login, restaurant search, menu browsing, cart calculation, ordering, payment, delivery assignment, tracking, cancellation, and administration.

Stage 4 – Code Quality

SonarQube analyzes bugs, code smells, vulnerabilities, duplicated code, maintainability, reliability, and test coverage. A failed quality gate blocks further deployment.

Stage 5 – Security

OWASP Dependency-Check, secret detection, and static security analysis are executed. Credentials are stored in GitHub Secrets rather than source code. Authentication and authorization are validated.

Stage 6 – Packaging

After successful checks, the application is packaged into a Spring Boot JAR. A Docker image is built and tagged with the Git commit SHA and pushed to the GitHub Container Registry.

Stage 7 – Development Environment

Successful CI automatically deploys the same artifact to Development for early integration testing, feature verification, and developer feedback.

Stage 8 – Testing/Staging Environment

The production-intended artifact is deployed to Staging. API, integration, smoke, UI, database, and order-processing tests are executed in a production-like environment. Production is blocked if these tests fail.

Stage 9 – Production Deployment

A Blue-Green deployment strategy minimizes downtime. Production requires successful build, testing, quality, security, and staging checks followed by manual approval.

Stage 10 – Monitoring

Prometheus and Grafana monitor application health, CPU/memory usage, response times, error rates, API availability, database connectivity, and critical order/payment failures.

6. Automated Testing Strategy

Following the test pyramid principle, fast unit tests run early for rapid feedback, while slower end-to-end tests execute later in the pipeline.

Test Level	Tool	Purpose	Trigger	Pipeline Action
Unit testing	JUnit	Validate isolated functions	Post-Build	Fail build if tests fail
Integration testing	Spring Boot Test	Validate module interaction	Post-Unit	Fail build if tests fail
API testing	Newman	Validate external endpoints	Staging Deploy	Block Production
UI testing	Selenium	Validate customer journeys	Staging Deploy	Block Production
Regression testing	JUnit/Newman	Ensure existing features still work	Nightly/Post-Build	Notify Team
Smoke testing	Bash/Curl	Check basic application health	Post-Deploy	Trigger Rollback
Security testing	OWASP	Detect vulnerabilities	Post-Build	Fail if Critical/High
Performance testing	JMeter	Test load capacity	Staging Deploy	Block Production

7. Deployment Strategy

The system uses three isolated environments with environment-specific configuration and secrets.

Environment	Purpose	Deployment Trigger	Testing Performed	Approval Requirement
Development	Feature validation and developer integration	Automatic on PR / branch update	Unit, Fast Integration	None (Automatic)
Staging	Production-like environment for final validation	Automatic on merge to main	E2E, API, Performance, Smoke	None (Automatic)
Production	Live user traffic	Post-Staging Success	Live Monitoring, Smoke	Manual Approval Required

8. Security Controls

Security is integrated throughout the pipeline. GitHub branch protection requires Pull Request reviews. GitHub Secrets prevent credential leakage. Dependency scanning and SAST identify vulnerabilities early. The architecture applies least-privilege access, HTTPS-only communication, strong authentication/authorization, secure minimal Docker images, environment-specific credentials, and audit logging.

9. Notification Strategy

GitHub Actions and Email notify the development team when builds succeed or fail, tests or quality/security gates fail, and staging or production deployments succeed or fail. Critical alerts are also generated when an automated rollback occurs.

10. Rollback Strategy

A deterministic rollback mechanism uses immutable Docker images. The flow operates as follows:

```
Production Deployment → Monitoring → Failure Detected → Stop/Redirect Traffic
                                                        ↓
Restore Previous Stable Version → Verify Health → Notify Team
```

Images are tagged with exact version SHAs, allowing the previous stable release to be restored quickly. Rollback conditions include high error rates, application unavailability, critical order/payment failures, database compatibility problems, or severe performance degradation.

11. Example GitHub Actions Workflow

```
name: CI/CD Pipeline
on:
  push:
    branches: [ "main" ]

jobs:
  build_and_test:
```

```
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Setup Java JDK
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Maven Build and Unit Tests
        run: mvn clean package
      - name: Code Quality & SonarQube
        run: mvn sonar:sonar -Dsonar.projectKey=food-delivery-system
      - name: OWASP Dependency Check
        run: mvn org.owasp:dependency-check-maven:check
```

11. Example GitHub Actions Workflow (Continued)

```
  docker_and_deploy_staging:
    needs: build_and_test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Build Docker Image
        run: docker build -t ghcr.io/org/food-delivery:${{ github.sha }} .
      - name: Push to GHCR
        run: |
          echo ${{ secrets.GHCR_PAT }} | docker login ghcr.io -u $GITHUB_ACTOR --password-stdin
          docker push ghcr.io/org/food-delivery:${{ github.sha }}
      - name: Deploy to Staging
        run: ./deploy-staging.sh ${{ github.sha }}
      - name: Run Staging Smoke Tests
        run: ./run-smoke-tests.sh staging

  production_deploy:
    needs: docker_and_deploy_staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Deploy to Production (Blue/Green)
        run: ./deploy-production.sh ${{ github.sha }}
      - name: Notify Team
        if: always()
        run: ./send-notification.sh ${{ job.status }}
```

12. CI/CD Workflow Explanation

```
Code Commit → Pull Request → Code Review → Build → Unit Tests → Integration/API Tests → Code Quality →
Security Scan → Package → Docker Image → Development Deployment → Staging Deployment → Smoke/E2E Tests →
Manual Approval → Production Deployment → Monitoring → Rollback if Necessary
```

The developer commits code, creating a Pull Request for peer review. After approval and merge, the build and automated tests run. SonarQube verifies quality and OWASP scans dependencies. Successful code is packaged into a

Docker image and deployed to Development and Staging. After staging validation and human approval, the image is released to Production. Monitoring detects anomalies and triggers rollback when necessary.

13. CI/CD Pipeline Decision Table

Condition	Action
Build fails	Stop pipeline
Unit test fails	Stop pipeline
Integration test fails	Stop pipeline
Quality gate fails	Stop pipeline
Security scan finds critical vulnerability	Stop pipeline
Staging deployment fails	Stop pipeline
Staging test fails	Stop pipeline
All checks pass	Allow production deployment (pending approval)
Production health check fails	Trigger rollback
Deployment successful	Continue monitoring

14. Risk and Mitigation

- Failed builds: Mitigated by local developer testing and pre-commit checks.
- Failed automated tests: Mitigated by failing the pipeline early, preventing broken code propagation.
- Security vulnerabilities: Mitigated by enforced dependency scanning and SAST tools.
- Deployment failure: Mitigated by immutable Docker images and Blue-Green deployment.
- Database migration failure: Mitigated by backward-compatible schemas and Staging validation.
- Configuration errors: Mitigated by secure, environment-specific secrets.
- Docker image issues: Mitigated by minimal, trusted base images.
- Production outage or payment failure: Mitigated by real-time monitoring and rapid rollback.

15. Benefits

The implementation of this CI/CD pipeline enables faster releases and reduces manual deployment errors. Early defect detection lowers the cost of fixing problems. Automated quality and security gates improve reliability and security. Consistent containerized deployments and rapid rollback reduce downtime, while continuous monitoring improves availability and supports coordinated development and operations for the Online Food Delivery platform.

16. Conclusion

In conclusion, the proposed CI/CD architecture establishes a robust, automated, and secure software delivery pipeline for the Online Food Delivery Management System. By integrating automated testing, code-quality analysis,

security scanning, controlled staging, Blue-Green deployment, monitoring, and rollback, the organization can deliver frequent updates without compromising service stability. The design supports reliable, secure, and highly available delivery of customer-facing features.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1
Requirement & Pipeline Analysis (15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear incomplete
Pipeline Architecture & Workflow (25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or p structured
Tools & Technology Selection (20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or identified
Automated Testing & Quality Integration (15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are or inappropriate
Deployment, Security & Rollback Strategy (15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is inc or lacks security considera
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incompl difficult to understand

Criteria

Criteria	Mark
Requirement & Pipeline Analysis (15%)	/5
Pipeline Architecture & Workflow (25%)	/4
Tools & Technology Selection (20%)	/4
Automated Testing & Quality Integration (15%)	/4
Deployment, Security & Rollback Strategy (15%)	/4
Pipeline Diagram & Documentation (10%)	/4
TOTAL	/25